

Guía de Aprendizaje: Modelo de Memoria en Hilos

Malak Sanchez

Monitorías de Sistemas Operativos

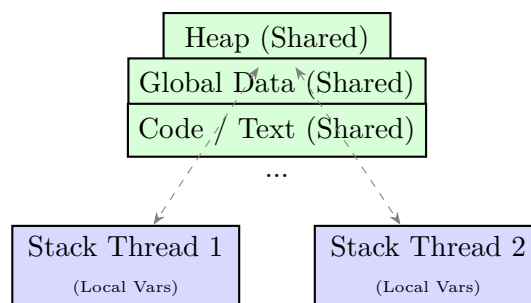
9 de mayo de 2026

1. Compartir vs Aislar

A diferencia de los procesos (donde cada uno tiene su isla"privada de memoria), los hilos viven en una casa compartida. Entender qué es privado y qué es público es vital para evitar errores desastrosos.

2. Mapa de Memoria

Todos los hilos de un mismo proceso comparten el direccionamiento virtual.



3. ¿Qué se comparte?

- **Variables Globales:** Si un hilo cambia una variable global, el otro verá el cambio inmediatamente.
- **Heap (malloc):** Si un hilo reserva memoria, el puntero es válido para todos los demás.
- **Descriptores de Archivo:** Si un hilo abre un archivo, otro puede leerlo usando el mismo FD.

4. ¿Qué es privado?

- **Stack:** Las variables declaradas dentro de la función del hilo son privadas.
- **Registros / PC:** Cada hilo está en su propia línea de código.

5. Ejemplo: La Variable Problemática

```
1  #include <stdio.h>
2  #include <pthread.h>
3
4  int global_shared = 0; // Public
5
6  void* worker(void* arg) {
7      global_shared++; // Modifying shared memory
8      return NULL;
9  }
10
11 int main() {
12     pthread_t t1, t2;
13     pthread_create(&t1, NULL, worker, NULL);
14     pthread_create(&t2, NULL, worker, NULL);
15     pthread_join(t1, NULL);
16     pthread_join(t2, NULL);
17     printf("Final value: %d\n", global_shared);
18     return 0;
19 }
```

6. Ejercicios

1. Modifique el ejemplo anterior para que cada hilo incremente la variable 1,000,000 de veces. ¿El resultado es siempre 2,000,000? ¿Por qué?
2. Intente pasar un puntero a una variable local del `main` a dos hilos diferentes. ¿Pueden ambos modificarla?